



Hotel Management System – Sprint Documentation

Version: 1.0.0

Prepared By: Development Team

Last Updated: {{today}}



Table of Contents

1. Sprint 1: Project Setup & Database Foundation
2. Sprint 2: Booking Module & Reservation System
3. Sprint 3: Guest Management System
4. Sprint 4: JavaFX Home + Booking Page
5. Sprint 5: Checkout + Room Status
6. Sprint 6: Housekeeping Module
7. Sprint 7: Staff Management
8. Sprint 8: Billing & Invoicing
9. Sprint 9: Finance Analytics
10. Sprint 10: Inventory Management
11. Sprint 11: QR-Based Smart Access
12. Sprint 12: Guest Experience & Feedback
13. Sprint 13: Multi-Hotel Support
14. Sprint 14: Website CMS + Media Upload
15. Sprint 15: Staff Attendance + Leave
16. Sprint 16: Biometric + IoT Room Control
17. Sprint 17: Report Export
18. Sprint 18: Global Settings & Permissions
19. Production Deployment Guide

☑ Sprint 1 – Step-by-Step Detailed Plan

Step #	Task Name	Component	Details	Output/Result
1	Install Java 17+ (OpenJDK)	Infrastructure	Download from https://adoptium.net and install. Configure JAVA_HOME.	java -version returns correct JDK
2	Install IntelliJ IDEA	Infrastructure	Install IntelliJ Community/Ultimate . Set up Maven/Gradle.	IDE ready for backend development
3	Install PostgreSQL 15+	Infrastructure	Download from https://www.postgresql.org/download/ . Set password. Install pgAdmin for GUI management.	Access to pgAdmin + CLI
4	Install Node.js 18+	Infrastructure	Download from https://nodejs.org . Install and verify with node -v.	Node + npm installed
5	Install Android Studio	Infrastructure	Set up Android SDK, emulator for React Native mobile testing.	Emulator ready for testing mobile app
6	Install Git	Infrastructure	Install Git, configure global username/email. Generate SSH key, add to GitHub.	git config, ssh -T git@github.com works
7	Initialize Git Repository	DevOps	Create GitHub repo hotelms-suite. Clone locally. Create branches: backend, desktop-ui, mobile-ui, website-ui, docs.	Branch structure in place, README committed
8	Initialize Spring Boot Project	Backend	Use https://start.spring.io . Add dependencies: Web, JPA, Security, PostgreSQL, Validation, Lombok, JWT. Use Gradle or Maven.	HmsBackend Application.java builds
9	Setup Folder Structure	Backend	Inside src/main/java/com/hms/: config, controller, service, repository, entity, dto, security, util.	Files placed in correct packages
10	Configure application.properties	Backend	Add DB config, JWT secret, JPA flags: spring.datasource.url, username, password, jwt.secret, jwt.expiration, server.port.	App reads and loads config properly

11	Create Database hms_db	Database	Run CREATE DATABASE hms_db; using pgAdmin or psql.	Empty DB created with admin access
12	Create Tables: users, hotels, rooms	Database	Define schema using SQL scripts. Use pgAdmin to execute. Set constraints and indexes.	Tables created in DB
13	Create Sample Data	Database	Insert sample hotel and admin user records for testing.	Initial test data available
14	Create Entity & Repository Classes	Backend	Classes: User, Hotel, Room, and corresponding Repository interfaces. Annotate with @Entity, @Table, @Id, etc.	Entities mapped and repositories functional
15	Configure Spring Security	Backend	Files: SecurityConfig.java, JwtUtil.java, JwtAuthenticationFilter.java. Add BCrypt encoder, JWT logic, exception handling.	Secure endpoints configured
16	Create AuthController & Service	Backend	Endpoints: - POST /api/auth/login - POST /api/auth/register - GET /api/auth/profile DTOs: LoginRequest, LoginResponse.	Working login/register via JWT
17	Create RoomController & Service	Backend	Endpoints: - GET /api/rooms - POST /api/rooms - PUT /api/rooms/{id} - DELETE /api/rooms/{id} - GET /api/rooms/available	Room management API functional
18	Add Swagger/OpenAPI Docs	Backend	Install springdoc-openapi-starter-webmvc-ui. Configure OpenApiConfig.java. Docs live at /swagger-ui.html, /v3/api-docs.	APIs visible with example payloads
19	Manual API Testing via Postman	Testing	Test all auth and room endpoints: ✓ 200 for valid calls ✓ 400/401 for failures ✓ Auth headers, JWT expiration	All endpoints tested and verified
20	Swagger UI Verification	Documentation	Open Swagger in browser. Check parameter descriptions, response codes, input types.	Visual API documentation complete

Sprint 2: Booking & Guest Management (Step-by-Step)

 **Duration: 5 Days**

 **Goal: Implement complete booking flow with guest linking and room availability logic**

Backend Tasks

Step #	Task Name	Details	Output/Result
B2-01	Create Guest Entity	Fields: id, name, phone, email, gender, id_type, id_number, address, created_at	Guest table created
B2-02	Guest Repository	GuestRepository extends JpaRepository<Guest, Long>	Guest repo supports save/find/delete
B2-03	Create Booking Entity	Fields: id, guest_id, room_id, check_in, check_out, services, total_price, created_at, status	Booking table created
B2-04	Booking Repository	BookingRepository extends JpaRepository<Booking, Long>	Booking repo implemented
B2-05	DTOs & Service Layer	Classes: CreateBookingRequest, UpdateBookingRequest, BookingDTO, service to handle logic	DTO conversion and business logic works
B2-06	Booking Controller	Endpoints: - POST /api/bookings - GET /api/bookings/{id} - PUT, DELETE etc.	Booking API available and secured
B2-07	Room Availability Filter	Logic to return rooms that are not booked during requested dates + status = EMPTY	Endpoint: /api/rooms/available returns data
B2-08	Booking → Room Status Link	Set room status to OCCUPIED when booking confirmed	Room grid reflects updated status
B2-09	Swagger Docs Update	Add guest and booking models with endpoint documentation	/swagger-ui.html updated

Desktop App (JavaFX) Tasks

Step #	Task Name	Details	Output/Result
--------	-----------	---------	---------------

D2-01	Booking Page Layout	Form fields: Guest info (name, phone, ID), check-in/out dates, room type, service dropdown, total calculator	Basic UI ready with alignment and theme
D2-02	Camera & ID Capture	Integrate OpenCV to capture guest photo; allow ID upload	Camera input stored in local/cloud
D2-03	Room Selector Integration	Show only available rooms (filter by date) using API	Dropdown shows correct filtered rooms
D2-04	Submit Booking Form	POST data to /api/bookings; show success popup on return	JavaFX app confirms booking
D2-05	Form Validation UI	Required fields, max length, ID format, phone format	Alerts for incorrect inputs

Website Frontend (JSP) Tasks

Step #	Task Name	Details	Output/Result
W2-01	/booking.jsp Form Design	Form: Guest details, check-in/out, room selector, services, price	Responsive Bootstrap-based UI page
W2-02	Room Availability in Form	AJAX call to backend /api/rooms/available based on selected dates	Room dropdown populates filtered list
W2-03	Submit Booking Form	Use Spring Form tag or JavaScript to post form to /api/bookings	Redirect to success page
W2-04	Success Page / Confirmation	Display guest name, booking dates, assigned room, total price	Confirmation screen shown
W2-05	Error Handling	Show form error on missing or invalid fields	Inline field error messages

Mobile App (React Native) – Future (Optional)

(No direct app booking planned for now; just referencing for future sprint)

Sprint 2 Deliverables

Area	Deliverable	Acceptance Criteria
Backend	Booking + Guest APIs	Endpoints work with validation

JavaFX App	Booking UI + API connection	Guest can be booked from desktop
JSP Website	Booking page + success confirmation	Website frontend booking flows
Documentation	Swagger updated	All new APIs documented

Sprint 2 Reference Links

Component	Documentation Link
Spring REST Controller	https://spring.io/guides/gs/rest-service/
JavaFX Form Design	https://openjfx.io
OpenCV for Java	https://opencv.org/releases/
Tesseract OCR	https://github.com/tesseract-ocr/tesseract
Spring Form in JSP	https://www.baeldung.com/spring-mvc-form-tutorial
Booking UI Design	https://mobbin.com/browse/web/apps/hotel-booking

✓ Sprint 3: Checkout & Invoice Flow

 Duration: 4 Days

 Goal: Implement room checkout logic, invoice generation (PDF), and QR-based payment confirmation.

Backend Tasks

Step #	Task Name	Details	Output/Result
B3-01	Checkout Entity (optional)	Add optional Checkout table or just update Booking with checkout_time, status (COMPLETED)	DB tracks completed bookings
B3-02	Update Booking Entity	Add fields: checkout_time, payment_mode, payment_status, invoice_url	Updated model fields in DB
B3-03	Create Checkout Service	Handle checkout logic: update booking, change room status to EMPTY, trigger invoice creation	Checkout API works with validation
B3-04	Checkout Controller Endpoint	API: POST /api/bookings/{id}/checkout	Room marked EMPTY, booking marked complete
B3-05	Generate PDF Invoice	Use iText or PDFBox to generate invoice with guest + stay details, taxes, and total	Downloadable invoice saved and URL returned
B3-06	Integrate UPI QR Code	Use QRGen or Google Chart API to generate QR code linked to UPI or Razorpay link	QR image returned on checkout
B3-07	Save Invoice to Storage	Store invoice file in local or Supabase Storage, save URL in DB	Invoice URL retrievable
B3-08	Swagger Documentation	Document new endpoints and schema changes	All updates visible in /swagger-ui.html

JavaFX Desktop App Tasks

Step #	Task Name	Details	Output/Result
--------	-----------	---------	---------------

D3-01	Checkout Page Layout	Display list of rooms marked as OCCUPIED with guest info	Table with checkout-ready rooms
D3-02	Checkout Form Fields	Fields: room no, guest name, check-in/out, services used, pending amount, payment mode dropdown	Checkout form with data pre-filled
D3-03	PDF Invoice Preview	Button: “Generate Invoice” → API call returns and opens PDF preview	Preview dialog or downloads PDF
D3-04	UPI QR Display	Display payment QR image returned from API	QR visible in UI
D3-05	Confirm Checkout	Button: “Checkout” → API POST call Room becomes empty, status reflected on dashboard	Room status updated, form reset
D3-06	Validation & Alerts	Show alert on unpaid dues, missing fields	AlertDialog or tooltip shown

JSP Website Frontend Tasks

Step #	Task Name	Details	Output/Result
W3-01	Checkout Page (Admin-Only)	Show table of current check-ins (rooms with guest, checkout due today)	Table with action buttons
W3-02	Checkout Summary Form	Details: guest name, room type, check-in/out, service charges, payment dropdown, generate invoice	Responsive form with summary
W3-03	Invoice Display & Download	Show PDF in embedded viewer or download button	Invoice PDF accessible to admin
W3-04	Payment QR Code View	Show QR generated from backend for UPI payment	Visible QR under payment section
W3-05	Confirm Checkout Submission	Submit form to backend, room marked as EMPTY, redirected to success page	Status updated, message shown

Mobile App (React Native) Tasks

Step #	Task Name	Details	Output/Result
A3-01	Payment Status View	Show list of rooms with pending checkout or unpaid dues	Admin can see real-time payment status

A3-02	PDF Invoice Viewer (Future)	Optional file viewer for downloaded invoices	View invoice in mobile PDF viewer
A3-03	Notifications (Future)	Show Toast or Push notification on checkout success	Guest/admin receives notification

✓ Sprint 3 Deliverables

Area	Deliverable	Acceptance Criteria
Backend	Checkout logic + PDF invoice + QR	APIs tested and return correct data
JavaFX	Checkout form + invoice + QR	Checkout reflected in UI and backend
Website (JSP)	Adminonly chckout + summary pg	Admin can process checkout from brow
Documentatio	Swagger API Docs	Updated for all new routes and models

🔍 Sprint 3 Testing Checklist

Test Case	Description	Expected Result
Checkout API Test	Call POST /bookings/{id}/checkout	Booking marked as completed
Room Status Update	Checkout completed → room should be EMPTY	Reflected in room grid
PDF Invoice Generation	API should return valid invoice	Invoice downloaded or shown
UPI QR Validity	API returns QR image correctly	QR scannable and links to payment
JavaFX Checkout Form	Submit button works, fields validate	Form submits + alerts shown
JSP Checkout Form	Admin uses form to complete checkout	Data saved, confirmation shown

📁 Sprint 3 Reference Links

Component	Link
PDF Generation (iText)	https://itextpdf.com/en/resources/examples
QR Code in Java	https://github.com/kenglxn/QRGen
Spring Boot File Upload	https://www.baeldung.com/spring-file-upload
UPI QR Code Spec	https://www.npci.org.in/what-we-do/upi/product-overview
JavaFX Invoice Viewer	https://openjfx.io

✓ Sprint 4 – Room Management Dashboard + Filters

 **Duration: 4 Days**

 **Goal: Build a unified room dashboard across all platforms with filtering, color-coded status, and CRUD operations.**

Backend Tasks

Step #	Task Name	Details	Output/Result
B4-01	Room Status Enum	Enum values: EMPTY, OCCUPIED, CLEANING, MAINTENANCE, DIRTY	Used across room logic and UI display
B4-02	Room Filter Endpoint	API: GET /api/rooms/filter Query Params: status, floor, type, AC/Non-AC, beds	JSON with filtered list of rooms
B4-03	Add Room Form Validation	Enforce unique room number, price range, floor > 0, etc.	Server-side form validation
B4-04	Update Room API	PUT /api/rooms/{id} with room update form: number, type, status, AC, floor, etc.	Room updated successfully
B4-05	Delete Room API	DELETE /api/rooms/{id}	Room deleted; returns success or constraint err
B4-06	Swagger API Update	Document all endpoints and enums	Swagger /rooms section updated

JavaFX Desktop Tasks

Step #	Task Name	Details	Output/Result
D4-01	Room Grid UI	3-column, 5-row scrollable grid of cards for rooms Each card shows: Room no., Type, Price, Status Color	Card layout with color-coded statuses
D4-02	Add Room Button & Form	Popup/modal with inputs: Room No., Type, Price, Beds, AC, Floor	Room saved via API
D4-03	Edit/Delete Options	Right-click or action menu on card: Edit (prefill form), Delete (with confirmation)	API call for edit/delete
D4-04	Filter Sidebar/Toolbar	Filters: Room status (checkbox), floor (dropdown), AC/Non-AC, Beds	Grid refreshes with filtered data
D4-05	Color Coding Legend	Display a color legend bar: Red = Occupied, Green = Empty, Orange = Maintenance, etc.	Clear user guidance

JSP Website Frontend Tasks

Step #	Task Name	Details	Output/Result
W4-01	/rooms.jsp Page	Show table view of rooms with colored status badges	Color-coded rows in table
W4-02	Room Filters (Client-side)	Filter form on top: status, floor, AC, beds Send AJAX request to /api/rooms/filter	Table updates based on selection
W4-03	Add/Edit/Delete Form (Admin)	Bootstrap modal form for add/edit + delete confirmation box	POST/PUT/DELETE triggered from JSP
W4-04	View-only Mode (Public Access)	Public users can see rooms but not edit	Only admin has control buttons

React Native Mobile App Tasks

Step #	Task Name	Details	Output/Result
A4-01	Room List Screen	FlatList of rooms with color status indicators	Card-style room list
A4-02	Apply Filters (Dropdown/Picker)	Filter: status (multi-select), floor, AC, beds	API call updates list
A4-03	Room Detail Modal	On tap, show modal with room info and action (edit/delete for admin roles only)	Full detail popup
A4-04	Admin Add/Edit Room (Optional)	Form with fields: number, type, floor, beds, price, AC, status	Form submits to backend API

Sprint 4 Deliverables

Area	Deliverable	Acceptance Criteria
Backend	Room CRUD + Filters	API returns filtered room list and handles edits
JavaFX App	Card-style room dashboard	Rooms color-coded and updated by filters
JSP Website	Admin table + public view page	CRUD modals functional for admin only
Mobile App	Room list + filter view	Admin can modify, others can view
Swagger Docs	Updated API documentation	/rooms endpoints documented fully

Sprint 4 Testing Checklist

Test Case	Description	Expected Result
Filter Rooms by Status	Select OCCUPIED, DIRTY filters	Only matching rooms shown
Add Room (Unique Number)	Add a room with an existing number	Returns validation error
Edit Room	Update room price/status	Values updated in DB
Delete Room	Remove room from system	Deleted or shows foreign key warning
Grid Refresh	Use filters and submit	Grid/Table updates live
Color Coding Consistency	Check status color codes	Color matches legend

Sprint 4 Reference Links

Component	Link
JavaFX Grid Layout	https://openjfx.io
Bootstrap Table + Modal	https://getbootstrap.com/docs/5.0/components/modal/
React Native FlatList	https://reactnative.dev/docs/flatlist
REST Room Filter Design	https://restfulapi.net/resource-filtering/

☑ Sprint 5 – Reservation System + QR & Notification

📅 Duration: 5 Days

🎯 Goal: Allow advance reservation, send confirmation via Email/WhatsApp, and generate secure QR codes for check-in access.

🔗 Backend Tasks

Step #	Task Name	Details	Output/Result
B5-01	Create Reservation Entity	Fields: id, guest_id, room_id, check_in_date, check_out_date, status (PENDING/CONFIRMED), token (UUID), created_at	Reservation table created in DB
B5-02	Reservation Repository	ReservationRepository extends JpaRepository<Reservation, Long>	CRUD ready for reservation
B5-03	Reservation Service Logic	Validate guest & room, prevent conflicts with existing bookings, generate token	Reservations processed correctly
B5-04	Create Reservation Controller	Endpoints: POST /api/reservations GET /api/reservations PUT /api/reservations/{id}/confirm	Reservation endpoints created
B5-05	Generate QR Code for Token	Use QRGen or similar library to generate QR for token linked to reservation	Image stored or returned in base64
B5-06	Send Email Notification	Use JavaMailSender or external service like SendGrid to email reservation confirmation with QR	Email sent with details + QR code
B5-07	WhatsApp Notification (optional)	Use Twilio WhatsApp API for sending message with reservation status & QR	WhatsApp message delivered
B5-08	Update Room Availability Logic	Exclude reserved rooms from available list during reservation window	Room availability respects reservations
B5-09	Swagger API Update	Add all /reservations routes + QR/notification flags	API documented in Swagger

JavaFX Desktop Tasks

Step #	Task Name	Details	Output/Result
D5-01	Reservation Page UI	Form fields: guest info, check-in/out date, room selector, status dropdown (PENDING/CONFIRMED)	Form created with proper layout
D5-02	Room Availability Calendar	Date range picker disables already booked or reserved rooms	No overlap allowed visually
D5-03	Submit Reservation Form	POST /api/reservations Show toast on success	Reservation saved + token returned
D5-04	QR Code Display	Show image of generated QR code + guest name after reservation	QR preview available
D5-05	Reservation List + Filter	List all current reservations by date/status Buttons: Confirm / Cancel	Table of reservations functional

JSP Website Tasks

Step #	Task Name	Details	Output/Result
W5-01	/reserve.jsp Page	Guest-facing form: name, ID, contact, dates, room type selector	Responsive Bootstrap form
W5-02	Validate Room Availability	Use AJAX to fetch available rooms for chosen date range	Dropdown updates in real time
W5-03	Submit Reservation	Submit to backend API; show confirmation page on success	Reservation saved
W5-04	Display QR Code + Download Option	After reservation, show QR image + download button	Guest sees QR + room info
W5-05	Send Email to Guest	Auto-send confirmation mail with QR and guest details using Spring Mail	Mail sent
W5-06	(Optional) WhatsApp Confirmation	Add backend switch for Twilio API; send message using guest mobile no	Message sent with QR (if enabled)

React Native App Tasks

Step #	Task Name	Details	Output/Result
A5-01	Reservation List for Manager	Show list of today's/this week's reservations with status and check-in date	Reservation dashboard visible
A5-02	Confirm/Cancel Reservation	Allow authorized user to confirm or cancel a reservation (PUT API call)	Status updated via mobile
A5-03	QR Scanner Preview (Future)	Prepare camera scanner to verify QR from reservation	Feature scoped for Sprint 11+

Sprint 5 Deliverables

Area	Deliverable	Acceptance Criteria
Backend	Reservation module + QR & notifications	Reservations created, QR stored, mail sent
JavaFX	Reservation screen + token + QR	QR & guest details shown after form submit
Website (JSP)	Public reservation form + email	Guest submits and receives QR
Mobile App	Reservation status dashboard	Staff views and confirms guest reservations
Swagger Docs	Reservation APIs documented	Confirmations visible in docs

Sprint 5 Testing Checklist

Test Case	Description	Expected Result
Reservation Creation	Submit reservation via API	Token generated + QR created
Email Confirmation	After booking, mail is sent	Guest receives email with PDF/QR
Room Exclusion Logic	Reserved rooms don't show in available list	Conflict prevented
JavaFX Form Submission	Submit form with valid data	Shows QR image and ID confirmation
QR Code Display in Web	Web page shows scannable QR code	Image appears and downloadable
WhatsApp Notification Test	Send message to test number (Twilio Sandbox)	Message received with booking info

Sprint 5 Reference Links

Component	Link
QRGen for Java	https://github.com/kenglxn/QRGen
Java MailSender Guide	https://www.baeldung.com/spring-email
WhatsApp via Twilio	https://www.twilio.com/docs/whatsapp/api
JSP Mail Integration	https://www.codejava.net/frameworks/spring/spring-email-tutorial
React QR Preview	https://github.com/awesomejerry/react-native-qr-code-scanner

✓ Sprint 6 – Staff Management & Role-Based Permissions

📅 Duration: 4 Days

🎯 Goal: Implement staff creation, editing, role-based access control (RBAC), and secure login UI for all platforms.

🔗 Backend Tasks

Step #	Task Name	Details	Output/Result
B6-01	Create Staff Entity	Fields: id, name, email, phone, gender, role (OWNER/MANAGER/STAFF), designation, hotel_id, is_active, created_at	Staff table created in DB
B6-02	Staff Repository + DTOs	Repository: <code>StaffRepository</code> extends <code>JpaRepository<Staff, Long></code> DTOs: <code>StaffDTO</code> , <code>CreateStaffRequest</code> , <code>UpdateStaffRequest</code>	Staff APIs usable with mapped models
B6-03	RBAC Role Enum Setup	Define Role Enum: OWNER, MANAGER, RECEPTIONIST, HOUSEKEEPING, etc.	Enum used in auth, JWT, UI access
B6-04	Assign Role to User at Signup	On user registration, assign role in DB and include in JWT claims	Role saved and embedded in token
B6-05	Secure API Access via Role	Use <code>@PreAuthorize</code> or custom security filters to restrict endpoints based on role	Unauthorized roles blocked automatically
B6-06	Staff Controller + APIs	Endpoints: <code>GET /api/staff</code> <code>POST /api/staff</code> <code>PUT /api/staff/{id}</code> <code>DELETE /api/staff/{id}</code>	Admin can manage all staff
B6-07	Login Role-Based Redirection	Decode JWT, return role in response → UI uses this for routing	Role returned on login
B6-08	Swagger API Update	Add staff routes with access restriction descriptions	Documented in Swagger

JavaFX Desktop Tasks

Step #	Task Name	Details	Output/Result
D6-01	Staff Management Page UI	Table of staff with name, role, contact, actions Role-specific permissions (only admin sees Add/Edit)	Table populated and filtered
D6-02	Add/Edit Staff Form (Modal)	Form: name, email, phone, gender, designation, role dropdown	New staff added or updated
D6-03	Delete Staff	Confirm dialog on deletion, API call → update table	Staff removed from system
D6-04	Role-Based View Filtering	Certain buttons/components hidden based on login role	Limited visibility for non-admins
D6-05	Auto Redirect after Login	After login, route to pages based on role: OWNER → Dashboard, STAFF → Tasks	Seamless flow post-login

JSP Website Tasks

Step #	Task Name	Details	Output/Result
W6-01	Staff Admin Panel (/staff.jsp)	Table view: Staff name, phone, role, active/inactive toggle, edit/delete buttons (admin only)	Admin-only table with actions
W6-02	Add Staff Form (Modal)	Form: name, phone, email, gender, role Submit via POST /api/staff	Form popup works and reflects changes
W6-03	Role-Based UI Elements	Use JSTL + session role logic to hide/show admin-only buttons	Restricted features visible only for managers/admin
W6-04	Login Redirect by Role	Use <code>session.setAttribute("role", user.getRole())</code> and redirect after login	Proper page per role

React Native App Tasks

Step #	Task Name	Details	Output/Result
A6-01	Staff List Screen (Admin View)	Only owner/manager roles see full staff list	List visible for managers only
A6-02	Add/Edit Staff Form	Optional form to add staff using role dropdown, phone, name	Mobile creation/edit API calls
A6-03	Restrict Module Visibility	Based on login role: hide dashboard modules for non-authorized users	Home screen customized per role
A6-04	Logout / Re-Auth	Role removed from context on logout; revalidated at login	Clean logout and relogin flow

Sprint 6 Deliverables

Area	Deliverable	Acceptance Criteria
Backend	Staff APIs + RBAC enforcement	Roles mapped to endpoints and visible in Swagger
JavaFX Desktop	Staff page + restricted actions	UI changes based on role
JSP Website	Staff management + conditional views	Manager-only buttons shown
Mobile App	Role-based home + admin screens	Screens change with login role
JWT Tokens	Token includes user role	Role extracted on frontend

Sprint 6 Testing Checklist

Test Case	Description	Expected Result
Add Staff (Admin only)	POST new staff from admin panel	Staff created
Edit Staff Info	Update phone/designation	Staff updated in DB
Delete Staff	Remove staff user	Staff no longer visible
Unauthorized Access	Staff tries to access restricted endpoint	403 Forbidden returned
Login Role Routing	Manager sees Dashboard, Reception sees Booking	Correct screen shown post-login
JWT Role Claims	Role present in token	Frontend reads and applies

Sprint 6 Reference Links

Component	Link
Spring RBAC Security	https://www.baeldung.com/spring-security-role-filtering
JWT Role Claim	https://jwt.io/introduction
JSP Role Logic	https://www.tutorialspoint.com/jsp/jsp-session-tracking.htm
React Role Context	https://reactrouter.com/en/main/start/tutorial#adding-protected-routes

☑ Sprint 7 – Housekeeping Module + Room Status Management

📅 Duration: 4 Days

🎯 Goal: Implement housekeeping request system, room cleaning flow, and allow staff to update room statuses through app and desktop.

🔑 Backend Tasks

Step #	Task Name	Details	Output/Result
B7-01	Create HousekeepingTask Entity	Fields: id, room_id, staff_id (optional), status (PENDING, ASSIGNED, COMPLETED), remarks, created_at, updated_at	Housekeeping task table in DB
B7-02	Housekeeping Repository	HousekeepingRepository extends JpaRepository<HousekeepingTask, Long>	CRUD operations for tasks
B7-03	Housekeeping Controller	Endpoints: POST /api/housekeeping GET /api/housekeeping PUT /api/housekeeping/{id}/status	Manager/staff can assign, update tasks
B7-04	Link Tasks to Room Status	On task completion, room status changes: PENDING → DIRTY COMPLETED → EMPTY or READY	Room status auto-updated
B7-05	Secure Staff Assignment	Only Manager role can assign or reassign tasks; staff can only see/update their assigned tasks	RBAC enforced on controller
B7-06	Add Housekeeping to Swagger Docs	All endpoints documented	/swagger-ui.html updated

💻 JavaFX Desktop Tasks

Step #	Task Name	Details	Output/Result
D7-01	Housekeeping Panel (Sidebar Page)	Table/grid showing tasks with room no., status, assigned staff, remarks	Visual overview of room statuses
D7-02	Create Task Modal	Form: Select room, add remark, assign staff (dropdown), default status = PENDING	Task created via API
D7-03	Update Task Status (Manager)	Dropdown to change task status: ASSIGNED, COMPLETED	Updates backend and room status
D7-04	Staff Assignment Logic	Dropdown to assign to logged-in staff or any available staff	Assignments reflected in DB

D7-05	Color Coding in UI	Status color mapping: Yellow = Cleaning Brown = Dirty ,Green = Empty,Orange = Maintenance	Dashboard consistent with legend
-------	--------------------	--	----------------------------------

JSP Website Tasks

Step #	Task Name	Details	Output/Result
W7-01	/housekeeping.jsp Page	Admin/Manager view of all tasks + filters by status	Paginated table or card-style UI
W7-02	Add Task Form (Admin Only)	Bootstrap modal: Room no, remark, assign to staff, initial status	Form working and updating API
W7-03	Mark Task Completed (Staff Panel)	Staff can login and see their tasks, mark as completed	Task status updated
W7-04	Access Restriction Logic	JSP tags check session role: only manager/admin sees full controls	RBAC works as expected

React Native Mobile App Tasks

Step #	Task Name	Details	Output/Result
A7-01	My Housekeeping Tasks Screen	Staff sees assigned tasks: Room no., remarks, status	Task list fetched from API
A7-02	Update Status (Mark as Done)	Staff updates status to COMPLETED; sends PUT /api/housekeeping/{id}/status	Status change reflected in system
A7-03	Role-Based Task View	Only staff with housekeeping role see this page	Menu item conditionally visible
A7-04	Manager Task Panel (Optional)	Managers view all tasks, assign/unassign from mobile	Same API as backend
A7-05	Color Tags / Badges	Use color-coded badges on task list based on status	Visual cue for quick scanning

Sprint 7 Deliverables

Area	Deliverable	Acceptance Criteria
Backend	Housekeeping APIs + room link logic	Tasks created, status updated, room syncs
JavaFX Desktop	Housekeeping panel + task editor	Staff update and manager assignment works

JSP Website	Role-based housekeeping view + updates	Staff can mark complete, managers can assign
Mobile App	Staff-only task screen + updates	App role-based views enabled
API Documentation	Swagger updated with new routes	Fully documented housekeeper endpoints

Sprint 7 Testing Checklist

Test Case	Description	Expected Result
Create Housekeeping Task	Create task for room	Room added to task list
Assign Task to Staff	Assign a staff member to a task	Assignment reflected
Update Status to COMPLETED	Staff updates task status	Task marked complete, room status updated
Unauthorized Role Access	Receptionist tries to assign task	403 error returned
UI Role Filtering (Web/App)	Staff vs Manager view	Pages/buttons hidden based on role
API Docs Testing	Swagger shows housekeeping endpoints	Correct params and models listed

Sprint 7 Reference Links

Component	Link
JavaFX Color Coding Grid	https://docs.oracle.com/javase/8/javafx/layout-tutorial
React Native Badge Styles	https://github.com/GeekyAnts/NativeBase
JSP Role Restriction	https://stackoverflow.com/questions/32471813/restrict-jsp-access
Spring Security Roles	https://www.baeldung.com/spring-security-method-security

☑ Sprint 8 – Billing & Invoicing System

📅 Duration: 4 Days

🎯 Goal: Build a centralized billing module for generating invoices, editing past bills, adding services, and payment tracking.

🔧 Backend Tasks

Step #	Task Name	Details	Output/Result
B8-01	Create Invoice Entity	Fields: id, booking_id, guest_id, room_id, invoice_number, date, service_details (JSON), total_amount, tax, discount, payment_status, pdf_url	Invoice table in DB
B8-02	Invoice Repository	InvoiceRepository extends JpaRepository<Invoice, Long>	API supports CRUD
B8-03	Billing Service	On checkout, system generates invoice (PDF + DB record), includes services & charges	Automated invoice generation
B8-04	Edit Historical Invoice Logic	PUT /api/invoices/{id} allows only admin to edit bill values, taxes, or notes	Past invoices editable via secure endpoint
B8-05	Payment Status Update Endpoint	PATCH /api/invoices/{id}/status to mark invoice as PAID, PENDING, CANCELLED	Payment state change tracked
B8-06	PDF Generation with Service Details	Use iText/PDFBox to generate full invoice with guest, room, service list, taxes, and totals	Clean, styled PDF invoice
B8-07	Attach Invoice to Booking	Booking now links to invoice_id for one-to-one mapping	Booking detail includes billing info
B8-08	Swagger API Update	Document all /invoices endpoints with request/response and roles required	Swagger updated

JavaFX Desktop App Tasks

Step #	Task Name	Details	Output/Result
D8-01	Invoicing Panel (Sidebar Entry)	List of generated invoices with filter by status, date, guest name	Grid display with filters
D8-02	View Invoice Button	View selected invoice details: guest, services, taxes, total Button to open/download PDF	Opens PDF in browser or downloads file
D8-03	Edit Invoice Form (Admin Only)	Form with editable fields: tax %, discount, add/remove service lines	Form submits via API and updates PDF
D8-04	Mark as Paid Button	Button on invoice row to update payment status (if PENDING)	Status badge updated on table
D8-05	Alert for Outstanding Payments	On login, show alert if invoices are unpaid or overdue	Real-time reminder for staff

JSP Website Tasks

Step #	Task Name	Details	Output/Result
W8-01	/billing.jsp Invoice Dashboard	Table: Invoice no., guest name, date, total, status, action buttons	Pagination and search bar added
W8-02	Invoice Viewer	View invoice details + PDF download or open in new tab	Clean layout for print or view
W8-03	Edit Invoice Modal (Admin Only)	Modal to update invoice data (discount/tax) and re-submit	Invoice and PDF regenerated
W8-04	Filter/Search by Status & Date	Filter bar on top: status (dropdown), date range (calendar picker)	Filter refreshes without page reload
W8-05	Update Payment Status	Manager marks invoice as PAID or PENDING using select or radio	Immediate backend update reflected

React Native App Tasks

Step #	Task Name	Details	Output/Result
--------	-----------	---------	---------------

A8-01	Invoice List Screen (Owner/Admin)	Show list of generated invoices by date, amount, and status	Scrollable FlatList
A8-02	View PDF Button	Button to view invoice using mobile PDF viewer (Expo FileSystem or react-native-pdf)	Invoices viewable in-app
A8-03	Mark Invoice Paid	For admins only – mark payment complete	API status patch successful
A8-04	Outstanding Payment Alert	Push or in-app alert showing unpaid invoices over 3 days old	Staff notified automatically

Sprint 8 Deliverables

Area	Deliverable	Acceptance Criteria
Backend	Invoice APIs + auto PDF generation	Booking generates invoice with itemized charges
JavaFX Desktop	Invoice list + edit + PDF preview	Admin can change tax/discount and regenerate
JSP Website	Billing table + filter + download PDF	Staff can manage and download bills
Mobile App	Invoice viewer + status toggler	Owner can view and confirm payments
API Docs	All billing endpoints documented	Swagger shows /invoices with auth roles

Sprint 8 Testing Checklist

Test Case	Description	Expected Result
Create Invoice on Checkout	Invoice created automatically with booking	Invoice ID returned

View Invoice PDF	View/download invoice from any UI	PDF content correct
Edit Invoice Amounts	Admin edits discount/tax	PDF and DB updated
Payment Status Update	Mark invoice as PAID	Status badge and backend reflect change
Filter Invoices by Status	Filter by PAID/PENDING	Correct list shown
RBAC on Invoice Editing	Non-admin edits invoice	403 error

Sprint 8 Reference Links

Component	Link
PDF Generation in Java	https://www.baeldung.com/java-pdf-creation
React Native PDF Viewer	https://github.com/wonday/react-native-pdf
Spring Boot Role Auth	https://www.baeldung.com/spring-security-role-filtering
JSP + Bootstrap Tables	https://getbootstrap.com/docs/5.0/content/tables/

Sprint 9 – Finance Analytics: Revenue, Expenses, Salary Dashboard

 **Duration: 4 Days**

 **Goal:** Implement finance tracking, summary widgets, filters, and charts for revenue, expenses, and salaries.

Backend Tasks

Step #	Task Name	Details	Output/Result
--------	-----------	---------	---------------

B9-01	Create FinanceEntry Entity	Fields: id, type (REVENUE/EXPENSE/SALARY), amount, date, department, note, created_at	Finance entry table created
B9-02	Finance Repository + DTOs	FinanceRepository extends JpaRepository<FinanceEntry, Long> + FinanceEntryDTO, filter DTOs	Ready for filter, summary, and add APIs
B9-03	Finance Controller & Endpoints	Endpoints: POST /api/finance GET /api/finance?type=&date=&dept= GET /api/finance/summary	Create entries, get filters, totals
B9-04	Department Filter Logic	Filter by department (Housekeeping, Reception, Kitchen, Management, etc.)	Filter returns correct grouped values
B9-05	Summary API for Cards	API: /api/finance/summary → returns totals: Revenue, Expenses, Salaries, Profit	Frontend widgets use this
B9-06	Monthly Trends Chart Data	API: /api/finance/trends?month=... returns monthly aggregated revenue and expenses for chart	Chart data generated
B9-07	Swagger API Update	Document all /finance routes, filters, summary endpoints	/swagger-ui.html updated

JavaFX Desktop Tasks

Step #	Task Name	Details	Output/Result
D9-01	Finance Dashboard Page (Sidebar)	KPI Cards: Total Revenue, Expenses, Salaries, Profit	Top of page – auto fetch from API
D9-02	Table of Entries	Columns: Type, Amount, Date, Department, Notes	Table with search, pagination
D9-03	Add Entry Form	Form: Select type, input amount, select department, optional notes, date picker	Adds new entry via POST
D9-04	Revenue/Expense Trend Chart	Line chart (monthly) with toggle for Revenue vs Expense	Visualized chart using JavaFX
D9-05	Filter Bar	Dropdowns: Type, Department; Date picker: from-to	Applies to table and cards

JSP Website Tasks

Step #	Task Name	Details	Output/Result
W9-01	/finance.jsp Analytics Page	Card Grid: Revenue, Expenses, Salaries, Profit	Top section loads summary from API
W9-02	Table of Entries	Responsive table with Type, Amount, Date, Department, Notes	AJAX-powered pagination and search
W9-03	Add Entry Modal (Admin Only)	Bootstrap modal to add financial entries	Form posts to /api/finance
W9-04	Filter by Date & Department	Dropdown filters + calendar picker (from-to), filtered via AJAX	Table + cards refresh without reload
W9-05	Expense Trend Chart	Chart.js or Google Charts integration showing expense vs revenue	Chart rendered from trend API

React Native App Tasks

Step #	Task Name	Details	Output/Result
A9-01	Finance Summary Cards	Show cards: Total Revenue, Expense, Salaries, Profit (Home/Admin Dashboard)	Cards styled with Flexbox
A9-02	Trend Chart Page (Optional)	Line/Bar chart of monthly revenue/expenses using Victory Native or react-native-chart-kit	Visual chart loaded from API
A9-03	Add Entry Form (Admin)	Form to submit finance data: type, amount, department, note, date picker	Posts to backend
A9-04	Filter Controls	Dropdowns and date range for filtering entries	Filters apply to card summaries & table

Sprint 9 Deliverables

Area	Deliverable	Acceptance Criteria
Backend	Finance APIs + summary + trends	Revenue/Expense APIs return correct totals & charts
JavaFX Desktop	Full finance page + filter + charts	KPI cards and graphs show accurate metrics
JSP Website	Finance page + table + modal form	All forms and charts respond to backend data
Mobile App	Cards + optional trend page	Owner sees financial overview
Documentation	Swagger Docs for finance endpoints	Complete with filtering examples

Sprint 9 Testing Checklist

Test Case	Description	Expected Result
Add Finance Entry	Submit REVENUE/EXPENSE via form	Entry saved and listed
KPI Cards Refresh	Total updates after new entry added	Revenue/Profit updated live
Monthly Trend API	Call trend endpoint for June	Monthly chart data returned
Filter by Department	Filter table/chart by KITCHEN or RECEPTION	Entries filtered correctly
Edit/Remove Entry (Admin)	Optional: edit or remove old entry	Changes reflected in totals
API Auth Check	Unauthorized role hits POST /api/finance	Returns 403 Forbidden

Sprint 9 Reference Links

Component	Link
-----------	------

JavaFX Line Chart	https://docs.oracle.com/javafx/2/charts/line-chart.htm
JSP Chart Integration	https://developers.google.com/chart
Spring Boot REST Filters	https://www.baeldung.com/spring-data-jpa-query
React Native Chart Kit	https://github.com/indiespirit/react-native-chart-kit

☑ Sprint 10 – Inventory Management + Stock Control

📅 Duration: 4 Days

🎯 Goal: Enable tracking of hotel inventory (items, categories, quantity), stock usage, restocking alerts, and role-based access.

🔧 Backend Tasks

Step #	Task Name	Details	Output/Result
B10-01	Create InventoryItem Entity	Fields: id, name, category, quantity, unit (kg/ltrs/pcs), threshold_limit, location, supplier_name, created_at, updated_at	Inventory table created
B10-02	Create InventoryLog Entity	Fields: id, item_id, action_type (ADD/USE/TRANSFER), quantity, note, user_id, timestamp	Usage log tracks stock changes
B10-03	Repositories + DTOs	Repositories for both tables + CreateItemRequest, UpdateItemRequest, InventoryLogDTO	Full API interaction
B10-04	Inventory Controller & Endpoints	Routes: GET /api/inventory POST /api/inventory PUT /api/inventory/{id} POST /api/inventory/log	Item CRUD + stock usage

B10-05	Low Stock Alert Endpoint	GET /api/inventory/alerts → returns items with quantity < threshold_limit	Used by frontend for warnings
B10-06	Swagger API Update	All inventory and usage logs documented	/swagger-ui.html updated

JavaFX Desktop App Tasks

Step #	Task Name	Details	Output/Result
D10-01	Inventory Panel UI (Sidebar)	Grid/List view of items: name, qty, unit, location, category, status indicator	Color badges for low stock
D10-02	Add/Edit Inventory Item Form	Modal form: name, unit, threshold, category, supplier, quantity	Create or edit via API
D10-03	Use/Reduce Inventory	Select item, input quantity used, note → API call to /inventory/log	Logs usage, updates main table
D10-04	Logs View with Filter	Table: item name, action type, quantity, user, timestamp Filter: date, type, item	Filtered inventory history
D10-05	Low Stock Alerts UI	Warning icons or row highlights if below threshold	Immediate visual alerts

JSP Website Tasks

Step #	Task Name	Details	Output/Result
W10-01	/inventory.jsp Page	Table of all inventory items Color-coded rows: Green = OK, Red = Low	Clean table with pagination
W10-02	Add/Edit Item Modal (Admin Only)	Bootstrap modal: item fields, unit, location, restock threshold	Data updates via AJAX
W10-03	Use Stock Modal	Form to reduce quantity on usage (room cleaning, kitchen, etc.)	POST to log usage
W10-04	View Usage Logs	Table of usage logs: who used, what, how much, when	Admin-only view

W10-05	Alerts Banner or Filter Option	Top alert: X items below threshold	Auto visible when below safety stock
--------	--------------------------------	------------------------------------	--------------------------------------

React Native App Tasks

Step #	Task Name	Details	Output/Result
A10-01	Inventory List (Read-Only for Staff)	Staff see inventory by category, search bar	Filter and scrollable list
A10-02	Add Item (Manager Only)	Form to add item from mobile: name, unit, qty, threshold	API call to backend
A10-03	Use Stock Button	Room staff uses items (e.g. 1 bedsheet → laundry)	Log created with staff role
A10-04	Alert Page (Manager Role)	Screen showing all items below threshold	Highlights critical stock

Sprint 10 Deliverables

Area	Deliverable	Acceptance Criteria
Backend	Inventory CRUD + Usage Logs + Alerts	API functional and data accurate
JavaFX Desktop	Item dashboard + log + alert UI	Complete interaction and visualization
JSP Website	Full inventory page + usage form	Staff and admin roles work differently
Mobile App	Inventory view + log use (for staff)	Controlled usage flow with visibility restrictions
Documentation	Swagger Docs for inventory APIs	Includes log + alert endpoints

Sprint 10 Testing Checklist

Test Case	Description	Expected Result
Add Inventory Item	Add with threshold, location	Added and saved in table
Log Stock Usage	Use item and reduce quantity	Log entry created, qty updated
Alert on Low Quantity	Set threshold < current value	Alert icon shows
View Inventory Logs	Logs filtered by item/date/type	Table returns correct records
Staff Role Restriction	Staff cannot add/edit inventory	403 Forbidden or buttons hidden
PDF/Excel Export (Optional)	Export logs for accounting	CSV downloaded (planned for future sprint)

Sprint 10 Reference Links

Component	Link
JavaFX Inventory Table Design	https://docs.oracle.com/javafx/2/ui_controls/table-view.htm
Inventory Management in JSP	https://getbootstrap.com/docs/5.0/components/table/
React Native Inventory Layout	https://reactnative.dev/docs/flatlist
Spring Boot REST Logs	https://www.baeldung.com/spring-boot-logging

☑ Sprint 11 – QR Access Integration (Encrypted Guest Entry System)

📅 Duration: 4 Days

🎯 Goal: Generate encrypted, time-limited QR codes post-reservation/booking for guest check-in access and verify them via secure backend endpoints.

🔧 Backend Tasks

Step #	Task Name	Details	Output/Result
B11-01	QR Token Generation Utility	Generate UUID token + AES-encrypt with expiry time (e.g., valid for 6 hours after check-in)	Secure encrypted string (Base64 or QR data)
B11-02	QR Token Entity (Optional)	Table: id, guest_id, booking_id, token, valid_from, valid_until, used (boolean), created_at	Token tracking for audit logs
B11-03	Link QR to Booking Flow	After booking or reservation confirmation, generate and store QR token	Token generated and returned
B11-04	QR Code Generation API	Use QRGen or Google Charts API to generate image from encrypted string	QR image returned to frontend
B11-05	QR Code Validation Endpoint	API: POST /api/qr/validate Decrypt token → check validity window → mark as used → return guest and room details	Only valid, unexpired tokens accepted
B11-06	RBAC on Validation	Only receptionist or access control devices can call validation endpoint	Secured via JWT or API key
B11-07	Swagger Documentation Update	Document /qr/validate and token structure, rules, and failure messages	Swagger reflects full QR flow

JavaFX Desktop Tasks

Step #	Task Name	Details	Output/Result
D11-01	Show QR Code on Reservation Confirm	After successful booking, show generated QR code image to print/send to guest	Visible on booking success screen
D11-02	QR Scanner Integration (Optional)	Use webcam + ZXing or OpenCV to scan QR and decode token	Validate against backend
D11-03	Validate QR on Check-in Page	Input scanned token manually or through camera → Send to <code>/api/qr/validate</code>	Guest check-in completes
D11-04	Failed Token Handling UI	Alert dialog for expired, reused, or invalid tokens	Real-time feedback to receptionist

JSP Website Tasks

Step #	Task Name	Details	Output/Result
W11-01	Show QR on Reservation Success Page	After guest reserves room, show QR with download button	QR visible to guest immediately
W11-02	Email QR Attachment	Send QR code as image or PDF to guest's registered email using MailSender	Guest receives QR after confirmation
W11-03	Mobile-Friendly QR Display Page	<code>/show-qr.jsp?id=booking_id</code> shows QR full screen (optional for printing or scanning at entrance)	Guest can re-access code anytime

React Native App Tasks

Step #	Task Name	Details	Output/Result
--------	-----------	---------	---------------

A11-01	Owner QR Viewer	Owner can view guest QR token generated for any room/guest (if needed)	Read-only view of tokens
A11-02	Mobile QR Scanner (Admin/Staff)	Use react-native-camera or expo-barcode-scanner to scan QR → call validate endpoint	Access granted or denied based on response
A11-03	Reissue QR (Owner Only)	Regenerate QR in case of token expiry (manually by owner/manager)	New token replaces old one
A11-04	QR Scan History Log (Optional)	Show access log (token used, time, by whom) for auditing	Displayed to authorized personnel only

Sprint 11 Deliverables

Area	Deliverable	Acceptance Criteria
Backend	Secure token system + validation endpoint	Encrypted QR, time-bound tokens, usage tracked
JavaFX Desktop	QR display on booking + optional validation	Guest QR shown and validated by reception desk
JSP Website	QR visible to guests + downloadable/email	Clean mobile-friendly QR with email support
Mobile App	QR scan & validate (Admin only)	Scan result + reissue ability for owners
API Docs	Documented QR endpoints + token rules	/qr/validate available in Swagger

Sprint 11 Testing Checklist

Test Case	Description	Expected Result
Generate QR after Booking	Guest books → QR token generated	Token returned and stored
QR Token Expiry	Try token after 6+ hours	Returns "Expired" message
Reuse QR Token	Use same token twice	2nd attempt rejected

Validate Token Flow	POST token to API	Returns guest + room if valid
QR Visible on Desktop/Website	After booking	QR displayed clearly
QR Scan Mobile	Scan with app	API returns access/denied

Sprint 11 Reference Links

Component	Link
Java AES Encryption	https://www.baeldung.com/java-aes-encryption
QRGen for Java	https://github.com/kenglxn/QRGen
react-native-camera Scanner	https://github.com/react-native-camera/react-native-camera
Email QR with Spring Mail	https://www.baeldung.com/spring-email
QR with JSP	https://developers.google.com/chart/infographics/docs/qr_codes

Sprint 12 – Guest Experience & Feedback System

 **Duration: 4 Days**

 **Goal:** Allow guests to submit feedback, raise complaints/service requests, and enable hotel staff/admins to respond and resolve them.

Backend Tasks

Step #	Task Name	Details	Output/Result
B12-01	Create Feedback Entity	Fields: id, guest_id, room_id, type (COMPLAINT/SERVICE/FEEDBACK), message, status (OPEN/CLOSED), priority, submitted_at, resolved_at	Feedback table created
B12-02	Create FeedbackRe	Fields: id, feedback_id, staff_id, message, timestamp	Tracks communication per complaint

	sponse Entity		
B12-03	Repositories + DTOs	Repositories for both tables + CreateFeedbackRequest, FeedbackResponseDTO, etc.	DTOs mapped to database
B12-04	Feedback Controller & Endpoints	Endpoints: POST /api/feedback GET /api/feedback?status=OPEN POST /api/feedback/{id}/response	Staff can respond, guest can submit
B12-05	Auto-Assign Priority (Optional)	Assign priority level based on type or keywords (Low/Medium/High)	Dynamic prioritization
B12-06	Notifications Integration (Optional)	On new response, notify guest via email/SMS/Push (based on settings)	Guest receives updates
B12-07	Swagger API Documentation	Document all /feedback and /feedback/response endpoints	API explorer reflects guest interaction module

JavaFX Desktop Tasks

Step #	Task Name	Details	Output/Result
D12-01	Guest Experience Page (Sidebar)	Table: All complaints, feedback, and requests; filters by status, type, date	Unified dashboard for management
D12-02	View Request Modal	View full message, room, guest, history of responses	Detail panel with scroll
D12-03	Respond to Complaint/Request Form	Form: message box, optional priority dropdown	Sends POST response to backend
D12-04	Close Ticket Button	Button to mark issue as resolved, status = CLOSED	DB updated, UI refreshed
D12-05	Filter Bar	Filters: status (OPEN/CLOSED), type (SERVICE/COMPLAINT), priority	Dynamic data grid filtering

JSP Website Tasks

Step #	Task Name	Details	Output/Result
W12-01	/feedback.jsp Form	Public form (guest-facing): Guest enters room no, mobile, message, category (dropdown)	Anonymous or semi-authenticated submission
W12-02	Confirmation Page	Shows submitted complaint/request ID and approximate resolution time	Clear feedback message shown
W12-03	Admin Feedback Panel	Staff login sees dashboard of open tickets with respond/close options	Actionable complaint table
W12-04	Response Thread Viewer	View all past replies (threaded view style or timeline)	Full feedback history

React Native App Tasks

Step #	Task Name	Details	Output/Result
A12-01	Guest Feedback Form	Guests submit service request, feedback, or complaint with dropdown + optional photo	Post request saved to backend
A12-02	Feedback History List	Guest sees list of submitted complaints + their statuses	List with filters
A12-03	Staff Dashboard (Manager View)	Managers/staff can view & respond to guest issues	Action list with resolution buttons
A12-04	Reply to Feedback	Form to respond to guest feedback with message and status change	Adds to response thread
A12-05	Push Notification for Response	Guests notified (Firebase/Expo) on response from staff	In-app notification displayed

Sprint 12 Deliverables

Area	Deliverable	Acceptance Criteria
Backend	Full feedback + response APIs	Guests submit and staff manage messages
JavaFX Desktop	Unified dashboard + complaint tracker	Staff can filter, respond, and close tickets
JSP Website	Guest request form + manager panel	Feedback saved and managed via web
Mobile App	Feedback interface + response tracking	Guests and staff use app to communicate
API Documentation	Swagger includes all routes	/feedback fully documented

Sprint 12 Testing Checklist

Test Case	Description	Expected Result
Guest Submits Feedback	Submit from app or website	Ticket created, visible to manager
Staff Responds to Complaint	Add message via dashboard	Message appears in guest view
Feedback Filtering	Filter by type, status, date	Table and list update
Close Ticket	Mark complaint as resolved	Status = CLOSED
Push/Email Notification	On response	Guest receives alert
View Feedback History	Timeline of replies shown	Chronological messages visible

Sprint 12 Reference Links

Component	Link
Spring Boot Messaging System	https://www.baeldung.com/spring-email
JSP Feedback Form Template	https://getbootstrap.com/docs/5.0/forms/layout

JavaFX Form Layout	https://docs.oracle.com/javafx/2/ui_controls/form.htm
React Native Feedback Screen	https://reactnative.dev/docs/scrollview
Firebase Notifications	https://rnfirebase.io/messaging/usage

✓ Sprint 13 – Multi-Hotel Support & Owner Dashboard

 Duration: 4 Days

 **Goal:** Enable support for managing multiple hotels under a single owner account, with a dashboard showing metrics, switching context, and ownership-level analytics.

Backend Tasks

Step #	Task Name	Details	Output/Result
B13-01	Extend Hotel Entity	Add fields if needed: region, registration_number, timezone, currency	Support diverse hotel profiles
B13-02	Link Users to Hotels	Modify User entity: hotel_id for staff, managed_hotels list for owners	Owner can access multiple hotels
B13-03	Create Hotel Switch API	GET /api/owner/hotels → list managed hotels POST /api/context/switch → set active hotel in session/context	Session-bound hotel switch
B13-04	Create Owner Dashboard Summary API	/api/owner/dashboard → returns combined revenue, occupancy, staff count per hotel	Aggregated multi-hotel metrics
B13-05	Role Guard Middleware Update	Ensure only OWNER roles can access cross-hotel dashboards	RBAC fully enforced
B13-06	Swagger Update	Document /owner/* APIs and hotel-switching logic	Available for testing via Swagger UI

JavaFX Desktop Tasks

Step #	Task Name	Details	Output/Result
D13-01	Hotel Selection Modal (OWNER only)	On login or top bar: dropdown/modal to switch between managed hotels	Desktop reloads on switch
D13-02	Dashboard Summary Cards	Total Rooms, Occupied Rooms, Staff Count, Monthly Revenue (for selected hotel)	Card values auto-refresh with context

D13-03	Multi-Hotel Visibility (Admin Tools)	Owner sees global tools (across hotels), manager sees single-hotel tools	Context-bound UI rendering
D13-04	Notifications for Hotel Selection Change	Notify user on successful context switch	Small toast or popup

JSP Website Tasks

Step #	Task Name	Details	Output/Result
W13-01	Owner Admin Panel (/owner.jsp)	Page shows list of all owned hotels with revenue, booking count, active staff	Summary page built with cards + tables
W13-02	Switch Hotel Context	Dropdown on header: current hotel label + switch menu	Hotel switch saves session and reloads
W13-03	Global Metrics Cards	Cards: Total Revenue, Avg Occupancy, Total Bookings across all hotels	Combined analytics
W13-04	Table of Hotel-wise Stats	Row: Hotel Name, Revenue, Occupancy %, Feedback Rating	Sorted/filtered table

React Native App Tasks

Step #	Task Name	Details	Output/Result
A13-01	Hotel Switch Dropdown (Owner Only)	Owner role sees dropdown to switch hotel context	Changes context in Redux/local storage
A13-02	Global Dashboard	KPIs: Revenue, Booking, Occupancy per hotel	Scrollable cards per hotel
A13-03	Active Hotel Context Indicator	Display selected hotel name on top of each module	Ensures correct context always visible

A13-04	Per-Hotel Action Panel (Optional)	Quick links to go to room list, staff, revenue for selected hotel	Contextually correct navigation
--------	-----------------------------------	---	---------------------------------

Sprint 13 Deliverables

Area	Deliverable	Acceptance Criteria
Backend	Multi-hotel model + switching system	Owner logs in, selects hotel, context switches
JavaFX Desktop	Owner context switcher + dashboard	Hotel-switch UI works and refreshes
JSP Website	Owner analytics and hotel switcher	Owner sees cross-hotel KPIs
Mobile App	Owner dashboard with multi-hotel switch	Owner monitors all hotels
Documentation	Swagger APIs for owner module	Fully documented /owner/*

Sprint 13 Testing Checklist

Test Case	Description	Expected Result
Owner Hotel Switch	Switch from Hotel A to Hotel B	All pages reload in Hotel B context
Owner Dashboard Data Accuracy	Total bookings/revenue match across hotels	Correct values rendered
Staff Hotel Isolation	Staff logged into Hotel X cannot access Hotel Y	Access denied
Mobile Hotel Context Switch	Dropdown changes app content to selected hotel	Navigation updates
Unauthorized Access Block	Manager trying to access /owner/ endpoints	403 Forbidden

Sprint 13 Reference Links

Component	Link
Multi-Tenant Architecture	https://dzone.com/articles/multi-tenancy-architecture-explained
JavaFX Role-Based Views	https://stackoverflow.com/questions/50414735/javafx-role-based-login
JSP Session Context Switching	https://www.javatpoint.com/jsp-session
React Native Context Manager	https://reactjs.org/docs/context.html

Sprint 14 – Hotel Website Content Manager + Media Upload

 **Duration: 4 Days**

 **Goal:** Enable hotel owners to manage public-facing website content including text sections, image uploads, room galleries, and homepage banners.

Backend Tasks

Step #	Task Name	Details	Output/Result
B14-01	Create WebsiteContent Entity	Fields: id, hotel_id, type (ABOUT, CONTACT, POLICY, BANNER, GALLERY), title, body, image_url, updated_at	CMS content model created
B14-02	File Upload Handler	REST endpoint using MultipartFile, store files on local/S3/Supabase Save public URL to DB	API returns uploaded image URL
B14-03	Website Content CRUD APIs	Endpoints: GET /api/website/content?type=ABOUT POST /api/website/content PUT /api/website/content/{id}	Content editable via APIs
B14-04	Image Gallery Controller	Upload/view/delete images for room types or banners (multiple files per section)	Room gallery API completed

B14-05	Public Content Endpoint	GET /public/website/{hotelId}/{type} returns published content for website integration	Website frontend uses dynamic backend content
B14-06	Swagger API Update	All website CMS and file APIs documented	/swagger-ui.html reflects new section

JavaFX Desktop Tasks

Step #	Task Name	Details	Output/Result
D14-01	Website CMS Panel (Sidebar)	Sectional editor: ABOUT, POLICY, CONTACT US with input fields and WYSIWYG-style text editor	Owners can update content
D14-02	Banner Image Upload Section	Drag-and-drop or button upload → Send file to API → Show preview	Banner images updated
D14-03	Room Image Gallery Editor	Upload multiple images per room type, preview thumbnails, delete option	API integration to gallery endpoints
D14-04	Preview Button	Preview current public page using stored backend data (opens in browser or embedded view)	Owner previews final website section

JSP Website Tasks (Public-Facing Website)

Step #	Task Name	Details	Output/Result
W14-01	Dynamic Home Page	Fetch /public/website/{hotelId}/BANNER + /ABOUT for intro section	Homepage displays content from backend
W14-02	Room Gallery Page	Load room images dynamically from backend via gallery API	Guests see room-type wise images
W14-03	About Us, Contact Us, Policy Pages	Load page content from backend CMS	Updated automatically from JavaFX/Desktop

W14-04	Image CDN Integration (Optional)	Serve images via Supabase Storage or local CDN	Fast media load times
--------	----------------------------------	--	-----------------------



React Native App Tasks

Step #	Task Name	Details	Output/Result
A14-01	CMS Editor (Owner Only)	Owner can update About, Contact, Policy from mobile via text fields + save button	POST/PUT to CMS API
A14-02	Mobile Banner Upload	Upload banner/room photos via gallery or camera	File uploaded, image visible
A14-03	Website Preview in WebView	Show embedded website preview (optional)	Opens mobile preview of public site



Sprint 14 Deliverables

Area	Deliverable	Acceptance Criteria
Backend	Website CMS content APIs + image upload	Website content editable per hotel
JavaFX Desktop	Content management UI with image handling	Owner updates site content with real-time preview
JSP Website	Dynamic content pages + gallery	Data fetched via CMS backend
Mobile App	Basic CMS editing + upload (owner only)	Works cross-platform (Expo or native)
Documentation	API docs updated for website CMS	/website/content, /gallery, /file-upload routes

Sprint 14 Testing Checklist

Test Case	Description	Expected Result
Update About Section	Submit new content from JavaFX or mobile	Website reflects change
Upload Room Images	Upload multiple photos under room category	Thumbnails visible
Banner Image API	Upload → fetch → display	Shows in homepage
View Dynamic Contact Page	/contact.jsp loads data from backend	Page displays live contact info
Mobile Preview (Optional)	View public page on app	Site loads as expected

Sprint 14 Reference Links

Component	Link
Spring File Upload	https://www.baeldung.com/spring-file-upload
Supabase Storage API	https://supabase.com/docs/guides/storage
JavaFX Image Upload UI	https://docs.oracle.com/javafx/2/ui_controls/image-view.htm
JSP Dynamic CMS	https://www.javatpoint.com/jsp-example-cms
React Native Image Upload	https://docs.expo.dev/versions/latest/sdk/imagepicker/

☑ Sprint 15 – Smart Staff Attendance + Leave Management

📅 Duration: 4 Days

🎯 Goal: Enable biometric/manual attendance logging, leave application and approval flow, staff scheduling, and shift tracking.

🔧 Backend Tasks

Step #	Task Name	Details	Output/Result
B15-01	Create Attendance Entity	Fields: id, staff_id, hotel_id, check_in_time, check_out_time, method (MANUAL/BIOMETRIC), shift, status (PRESENT/ABSENT)	Records daily attendance
B15-02	Create LeaveRequest Entity	Fields: id, staff_id, from_date, to_date, reason, status (PENDING/APPROVED/REJECTED), applied_on, approved_by	Tracks staff leave requests
B15-03	Attendance & Leave Repositories	Add JPA Repos + DTOs: AttendanceDTO, LeaveRequestDTO, CreateLeaveRequest	CRUD-ready
B15-04	Attendance Controller APIs	POST /api/attendance/checkin, /checkout, GET /staff/{id}/attendance?date=	Clock-in/out and daily reports
B15-05	Leave Request APIs	POST /api/leave, PUT /api/leave/{id}/status, GET /leave/staff/{id}, GET /leave/pending	Staff apply, admin approve
B15-06	Shift Management (Optional)	Add logic to associate staff roles with shift windows (e.g., 8AM–8PM) and validate time limits	Used for alerts and policy checks
B15-07	Swagger Documentation Update	Document all routes /attendance/*, /leave/*, /shifts/*	Fully available via Swagger

JavaFX Desktop Tasks

Step #	Task Name	Details	Output/Result
D15-01	Staff Attendance Panel (Sidebar)	Table: Staff Name, Check-in, Check-out, Status, Method	Updated daily per user
D15-02	Manual Check-in/Check-out Button	Allow admin to manually check in/out staff (e.g., in case of biometric failure)	Calls POST /api/attendance/checkin
D15-03	Leave Requests Table	View all submitted leave requests with filters by date, staff, status	Approve/Reject buttons with comment box
D15-04	Attendance Calendar View (Optional)	Visual calendar showing attendance marks: Green = Present, Red = Absent	Clear visual insights
D15-05	Auto Reminder for Pending Leaves	Highlight pending approvals and staff without attendance today	Alerts in dashboard

JSP Website Tasks

Step #	Task Name	Details	Output/Result
W15-01	Staff Portal: Attendance Table	Daily record of login/logout times, status	Fetches from backend
W15-02	Leave Application Form (/leave.jsp)	Fields: From Date, To Date, Reason Button: Submit	Submits to leave API
W15-03	Admin Leave Approval Dashboard	Table with filters: staff name, status, date range Action buttons: Approve, Reject	Leave status changes
W15-04	Attendance Report Download (Optional)	Export monthly attendance as CSV or PDF	Integrated with finance

React Native App Tasks

Step #	Task Name	Details	Output/Result
--------	-----------	---------	---------------

A15-01	Check-In/Out Button (Staff Role)	Button on dashboard for staff to check in/out; saves geolocation if enabled	Status recorded via API
A15-02	View Attendance History	List of dates, with status, check-in/out times	Scrollable UI
A15-03	Submit Leave Request	Form with From/To date, reason, submit button	API: /leave
A15-04	Leave Status Tracker	See applied leaves and their approval status	Real-time update via API
A15-05	Admin Approval (Owner/Manager)	Manager role sees pending requests and can approve/reject	Managed from mobile

Sprint 15 Deliverables

Area	Deliverable	Acceptance Criteria
Backend	Full attendance + leave modules	Time tracking + leave logic complete
JavaFX Desktop	HR-style panel for attendance and leave	Admins control attendance, leave, and shifts
JSP Website	Leave request portal + approval table	Accessible to staff and managers
Mobile App	Check-in/out, request leave, view attendance	Role-based access, clean UX
Documentation	Swagger for all HR routes	/attendance, /leave, /shifts well documented

Sprint 15 Testing Checklist

Test Case	Description	Expected Result
-----------	-------------	-----------------

Staff Check-in	Click button on app or desktop	Time logged
Leave Request Submit	Enter date range and reason	Request created with status = PENDING
Admin Approves Leave	Manager approves request	Status = APPROVED
Prevent Double Check-in	Same staff tries to check-in twice	API blocks or updates record
View Attendance Calendar	Admin sees staff calendar view	Shows accurate data
Filter Leave Records	Search by staff/date/status	Filtered results shown

Sprint 15 Reference Links

Component	Link
Spring Boot Attendance API	https://www.baeldung.com/spring-data-rest-controllers
JavaFX Calendar View	https://github.com/JFXtras/jfxtras-labs
JSP Form Processing	https://www.javatpoint.com/jsp-form-processing
React Native Time Picker	https://github.com/react-native-datetimepicker/datetimepicker
Mobile Geolocation (Optional)	https://docs.expo.dev/versions/latest/sdk/location/

Sprint 16 – Biometric Integration & Smart Room Automation Setup

 Duration: 4–5 Days

 **Goal:** Integrate biometric attendance and enable groundwork for IoT-based smart room automation (e.g. AC, lights, smart locks) using secure APIs and device registration.

Backend Tasks

Step #	Task Name	Details	Output/Result
--------	-----------	---------	---------------

B16-01	Add DeviceRegistry Entity	Fields: id, device_id (MAC/UUID), room_id, device_type (AC/LOCK/LIGHT/CAMERA), status, registered_at	Registry for all connected IoT hardware
B16-02	Biometric Device API	API endpoint: POST /api/biometric/checkin Input: biometric_device_id, staff_id Save attendance via biometric method	Attendance integrated from scanner
B16-03	Room Automation API	API endpoint: POST /api/room/{id}/control Input: device_type, action (ON/OFF/LOCK/UNLOCK)	Triggers device command
B16-04	Device Authorization Logic	Only allow trusted device IDs to trigger or receive updates for a room	IoT device whitelist implemented
B16-05	IoT Token Communication (Optional)	Secure room device communication using bearer tokens or keys	Future-proof setup for hardware
B16-06	API Rate Limiting/Security Filter	Ensure IoT APIs are protected from abuse with token, IP, or device ID restriction	Security layer added
B16-07	Swagger Update	Add /biometric and /room/control documentation	API testing ready

JavaFX Desktop Tasks

Step #	Task Name	Details	Output/Result
D16-01	Biometric Config Panel	Table: registered biometric devices (ID, staff, active?) Button to assign or deregister device	Device registry UI
D16-02	Smart Room Control Panel	Show rooms with connected devices (AC, Lights, Locks) → control via toggle buttons	Sends API commands
D16-03	Live Device Status Indicator	Show status of room devices (AC=ON, LOCKED etc.) based on last sync	Updated every 10 sec (polling or push)

D16-04	Device Control Log View (Optional)	Table: action, device, time, by whom	Staff auditing panel
--------	--	--------------------------------------	----------------------

JSP Website Tasks

Step #	Task Name	Details	Output/Result
W16-01	Smart Room Status Viewer (Public/Owner)	Display whether room is ready, AC is ON, or auto-lock is active (read-only)	Real-time guest experience
W16-02	Biometric Setup Page (Admin only)	Admin assigns biometric devices to staff ID via dropdown	Updates device registry
W16-03	Room Automation Controls (Optional Admin)	Staff can send lock/unlock or AC toggle commands from web	API to backend IoT bridge

React Native App Tasks

Step #	Task Name	Details	Output/Result
A16-01	Smart Room Control Panel (Manager Only)	Screen with toggle switches for AC/Lights/Lock by room	Sends API request to /room/control
A16-02	View Device Status	Read-only view showing status of smart devices per room	Updated using polling or socket
A16-03	Biometric Status Page	Staff sees if their biometric check-in was successful and device used	Verifies attendance success
A16-04	Alert for Unauthorized Devices	If a device is not authorized → show error	Extra security at staff level

Sprint 16 Deliverables

Area	Deliverable	Acceptance Criteria
------	-------------	---------------------

Backend	Biometric + Smart Room APIs	Device registry + control working
JavaFX Desktop	Biometric config + smart control dashboard	Staff mapped and room status visible
JSP Website	Owner panel + device assignment view	Admin manages biometric access
Mobile App	Control + status screens for owners/managers	Cross-platform smart management
Documentation	Swagger for /biometric + /room/control	Token & device ID auth shown

Sprint 16 Testing Checklist

Test Case	Description	Expected Result
Register Biometric Device	Assign device to staff	Device linked
Biometric Attendance API	Trigger check-in via device	Attendance saved
Toggle Room Light/AC from App	Manager toggles light	Room responds + status updated
Unauthorized Device API Access	Invalid device calls /room/control	403 Forbidden
Device Registry Security	Prevent unknown MAC/device_id from acting	System blocks request
View Smart Status from Desktop	View all rooms' AC/light statuses	Accurate and live

Sprint 16 Reference Links

Component	Link
Java Biometric Device Integration	https://github.com/bayazit/biofinger
IoT Device Control via API	https://developer.mozilla.org/en-US/docs/Web/HTTP/IoT
JavaFX IoT Panel UI	https://docs.oracle.com/javafx/2/ui_controls/toggle-button.htm
React Native IoT Integration	https://github.com/react-native-webrtc/react-native-push-notification
Biometric with Spring Boot	https://dzone.com/articles/biometric-authentication-spring-security

Sprint 17 – Reports Module + Export to PDF/Excel

 Duration: 4 Days

 Goal: Build a central reporting system to generate exportable PDF and Excel reports for bookings, finance, housekeeping, inventory, and staff attendance.

Backend Tasks

Step #	Task Name	Details	Output/Result
B17-01	Define Report Types	Booking Report, Revenue Report, Expense Report, Inventory Report, Attendance Report	Clearly mapped endpoints for each report
B17-02	Report Controller Endpoints	Endpoints: GET /api/report/booking?dateRange=. .. GET	Supports filters and file format params

		/api/report/finance?month=... GET /api/report/attendance?staffId=...	
B17-03	PDF Report Generation Utility	Use iText or Apache PDFBox to generate PDF with tables, headers, logos, and footers	Styled, exportable PDF
B17-04	Excel Report Generation Utility	Use Apache POI for creating downloadable .xlsx files	Formatted Excel with auto-width and filters
B17-05	Filter Logic with Params	Accept parameters for filtering reports: date, status, staff, hotel, type, etc.	Relevant data returned only
B17-06	Secure Export Actions	Only OWNER/MANAGER roles allowed to generate/export reports	Role checks enforced
B17-07	Swagger Documentation Update	Add /api/report/* with file formats, filters, and headers	Full API testable via Swagger

JavaFX Desktop Tasks

Step #	Task Name	Details	Output/Result
D17-01	Reports Panel (Sidebar)	Dropdowns: Report Type, Filter By (Date, Staff, Dept), Format (PDF, Excel) Button: "Generate Report"	Unified export dashboard
D17-02	Booking Report Export	Filter by room, status, guest, date	Table + export buttons
D17-03	Finance Report Export	Revenue, Expense, Salary – grouped by department or date	Chart + downloadable file
D17-04	Staff Attendance Export	Staff-wise/monthly attendance grid export	Calendar or matrix format
D17-05	View Generated Files History (Optional)	Table showing past report exports with download links (stored locally)	Reusable without re-generation

JSP Website Tasks

Step #	Task Name	Details	Output/Result
W17-01	/reports.jsp Page	Tabs or dropdown: Booking, Finance, Inventory, Staff	Separate reports with specific filters
W17-02	Date Range + Filter UI	From-To pickers, department/staff selectors	AJAX loads filtered preview
W17-03	Export Buttons (PDF/Excel)	Buttons to download in either format	Calls backend and triggers download
W17-04	Download Links	Success messages and file links on completion	Clear confirmation

React Native App Tasks

Step #	Task Name	Details	Output/Result
A17-01	Mobile Report View (Manager/Owner)	Filter controls: Report Type, Date Range Option to request PDF/Excel	View + download PDF
A17-02	Export History (Optional)	List of recent reports generated with re-download option	History feature added
A17-03	Secure Role Check	Only Admin/Owner roles access export feature	Proper RBAC check

Sprint 17 Deliverables

Area	Deliverable	Acceptance Criteria
Backend	Report APIs + PDF/Excel export	Multiple reports generated via secure endpoints
JavaFX Desktop	Unified report generation UI	Admin can filter and export from desktop

JSP Website	Staff-accessible report page	PDF and Excel downloads available
Mobile App	Owner/Manager export panel	Reports downloadable from app
Documentation	Swagger docs + format options	/report/* documented with all params

Sprint 17 Testing Checklist

Test Case	Description	Expected Result
Booking Report (PDF)	Generate report for last 7 days bookings	File downloads, shows expected data
Staff Attendance Export	Monthly export by staff	XLS or PDF with calendar view
Invalid Export Format	Request unsupported file type	Returns error 400
Unauthorized Access	Staff tries to access finance report	403 Forbidden
View Downloaded Files	File opens with correct layout and headers	Logo, totals, structured data shown
Export via App	Manager generates booking report	File viewable and sharable from app

Sprint 17 Reference Links

Component	Link
Java PDF Export (iText)	https://itextpdf.com/en/resources/books/itext-7-building-blocks
Apache POI for Excel	https://poi.apache.org/spreadsheet/index.html
JavaFX Report UI Patterns	https://docs.oracle.com/javafx/2/ui_controls/tables.htm
React Native File Handling	https://github.com/rnmods/react-native-document-picker
JSP File Download	https://www.javatpoint.com/jsp-download-file-example

✔ Sprint 18 – System Settings, Roles & Final Integration Testing

📅 Duration: 4–5 Days

🎯 Goal: Finalize global configurations, assign detailed access permissions, centralize settings (branding, hotel policies, tax configs), and conduct full system testing across all platforms.

🔧 Backend Tasks

Step #	Task Name	Details	Output/Result
B18-01	Global Settings Entity	Fields: id, hotel_id, key (e.g., TAX_RATE, LOGO_URL, CHECKIN_POLICY), value, updated_at	Flexible setting model
B18-02	System Settings API	GET /api/settings, POST /api/settings, PUT /api/settings/{id}	Admin can configure policies, app settings
B18-03	Role Permissions Config (RBAC Matrix)	Use RolePermission table: id, role, module, access_type (READ/WRITE/EDIT/DELETE)	Dynamic access layer
B18-04	Permissions Middleware	Validate API access using permission entries instead of static roles	Modular and scalable security
B18-05	Hotel Branding API	Allows logo, primary color, contact info upload (stored in Global Settings table)	Branding synced to all UI platforms
B18-06	Swagger Final Update	Review all API tags, group endpoints, document authorization headers and example tokens	Finalized docs for clients and testers

JavaFX Desktop Tasks

Step #	Task Name	Details	Output/Result
D18-01	System Settings Panel (Sidebar)	Cards/Fields: Tax %, Hotel Logo Upload, Default Policies, GST config	Updates reflected across UI
D18-02	Role Permissions Editor (Admin Only)	Editable matrix/grid: Roles × Modules × Actions	Role-based control activated
D18-03	Branding Sync UI	Preview how logo/colors appear in app post-update	Live preview or next-launch refresh
D18-04	Notification Settings (Optional)	Enable/disable push, email alerts per role	Global toggles

JSP Website Tasks

Step #	Task Name	Details	Output/Result
W18-01	Admin Settings Panel	Fields: Website title, footer info, logo, email config, policies	Dynamic website branding
W18-02	Manage Permissions (Manager/Owner)	Table of roles vs access → modify who can access what (for website modules only)	Controls what staff sees
W18-03	Real-time Branding Display	Website changes based on backend values (logo, tagline, etc.)	Instant reload

 React Native App Tasks

Step #	Task Name	Details	Output/Result
A18-01	Owner Settings Panel	Update hotel-wide configs (check-in times, tax rate, contact info, etc.)	App-wide changes sync across devices
A18-02	App Theme Branding Sync	Sync brand colors and logo to header, splash, and footer	Branded look matching desktop and web
A18-03	Push Notification Config	Toggle notifications (optional) for bookings, services, complaints	Controlled by global settings

 Sprint 18 Deliverables

Area	Deliverable	Acceptance Criteria
Backend	Settings + Role Matrix + Branding APIs	Configurable policies and granular access
JavaFX Desktop	Admin system config + role permission UI	Role access enforced visually and functionally
JSP Website	Website branding + module permissions	Live site reflects admin edits
Mobile App	App-level customization and settings	Owner can control appearance and alerts
Documentation	Final Swagger + Setup Docs	Ready for deployment handoff

 Sprint 18 Final Testing Checklist

Test Case	Description	Expected Result
-----------	-------------	-----------------

Update Tax % in Settings	Change in settings reflects in invoice calculations	Correctly applied everywhere
Restrict Booking Access for Role	Remove booking access for housekeeping role	Page/API blocked
Change Branding Logo	Upload new logo from panel	Visible on website, app, and desktop
Create Custom Policy	Add key: CHECKIN_GRACE_TIME, value: 30	Fetches and shown on guest interface
Push Toggle	Turn off push notifications for staff	No notifications sent
Validate Swagger & Docs	Check Swagger grouping and tags	API sections logically grouped

Sprint 18 Reference Links

Component	Link
RBAC Implementation with DB	https://www.baeldung.com/spring-security-role-filtering
Global Settings in Spring Boot	https://www.baeldung.com/properties-with-spring
JavaFX Branding	https://docs.oracle.com/javafx/2/api/javafx/scene/layout/Pane.html
React Native App Theming	https://reactnative.dev/docs/style
JSP Dynamic Branding	https://www.javatpoint.com/jsp-application-implicit-object

Production Deployment Plan

Phase	Task	Description	Responsible	Tools/Commands
✅ 1. Pre-Deployment	Freeze Code	All branches frozen, final QA on main/release	Dev Team	GitHub branch protection

	Environment Setup	Setup production server: VPS (e.g. DigitalOcean, EC2), PostgreSQL instance, Tomcat (for JSP)	DevOps	apt, systemctl, .env files
	Secrets & Env Config	Create and securely store environment variables (.env) for DB, JWT, email	DevOps	GitHub secrets / environment.yaml
✅ 2. Backend Deployment	Build & Deploy Spring Boot	Use mvn clean install, deploy JAR on VPS or container	Backend Lead	java -jar hms-backend.jar
	Configure PostgreSQL	Create schema, run migrations (schema.sql/data.sql)	DBA	psql -U postgres -d hms_prod
✅ 3. Frontend Deployment	JavaFX Desktop	Package JAR and installer (.exe or .dmg), distribute via internal channel	Desktop Lead	Launch4J / JPackage
	JSP Website	Deploy WAR to Tomcat or static files on server	Web Dev	scp, tomcat/webapps/
	React Native App	Build APK (Android) and IPA (iOS) via Expo or EAS	Mobile Lead	eas build -p android
✅ 4. Data & Integration	Hotel & Staff Seed Data	Load initial records: hotels, users, room types	Admin Team	API/SQL
	API Key Setup	For QR, Biometric, Channel Manager, etc.	DevOps	API gateway / header tokens
✅ 5. DNS & Domain	Domain Linking	Link domain to frontend (website) and backend endpoints	DevOps	Cloudflare / DNS
✅ 6. Launch	Go-Live	Final testing, announce launch	Product Owner	Status announcement

Post-Launch Monitoring Checklist

Area	What to Monitor	Tool/Process	Alert?
✅ API	Latency, 500 errors, response time	Postman Monitor / New Relic / Grafana	✅ Email/Slack

✓ Database	Slow queries, size, connection pool	pgAdmin / pgBadger / APM	✓
✓ Mobile App	Crash logs, update adoption	Firebase Crashlytics	✓
✓ Website	Page load, API integration	GTMetrix / Google Analytics	—
✓ Security	Unauthorized access, failed logins	Spring Security Logs / Fail2Ban	✓
✓ Logs	Log rotation, error spikes	ELK Stack / Papertrail	✓
✓ Backups	Auto DB & file backups	pg_dump / cron jobs / AWS S3	✓ Daily
✓ Uptime	Server ping + endpoint tests	UptimeRobot / Pingdom	✓